2023 IEEE/ACIS 21st International Conference on Software Engineering Research, Management and Applications (SERA) | 979-8-3503-4588-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/SERA57763.2023.10197790

Multi-level Adaptive Execution Tracing for Efficient Performance Analysis

Mohammed Adib Khan
Brock University
St. Catharines, ON L2S 3A1
ak19qp@brocku.ca

Naser Ezzati-Jivan
Brock University
St. Catharines, ON L2S 3A1
nezzati@brocku.ca

***Abstract*—Troubleshooting system performance issues is a challenging task that requires a deep understanding of various factors that may impact system performance. This process involves analyzing trace logs from the kernel and user space using tools such as ftrace, strace, DTrace, or LTTng. However, pre-set tracing instrumentation can lead to missing important data where not enough components of the system include observability coverage. Also, having too much coverage may result in unnecessary noise in the data, making it extremely difficult to debug. This paper proposes an adaptive instrumentation technique for execution tracing, which dynamically makes decisions not only for which components to trace but also when to trace, thus reducing the risk of missing important data related to the performance problem and increasing the accuracy of debugging by reducing unwanted noises. Our case study results show that the proposed method is capable of handling tracing instrumentation dynamically for both kernel and application levels while maintaining a low overhead.**

***Index Terms*—Software observability, software performance, adaptive instrumentation, change detection, execution tracing**

I. Introduction

Software developers use various static and dynamic analysis [1], [2] methods such as logging [3], tracing [4], [5], and monitoring key metrics [6], [7] to analyze and comprehend their programs, and diagnose and resolve performance issues and faults.

The traditional approach in tracing and logging is to maintain a fixed configuration throughout the entire execution, even though the behavior of the system may change completely [5]. Determining which tracepoints or probes to enable is a major challenge in tracing and logging. If too many tracepoints are enabled, the resulting data can be cluttered with irrelevant information, making it difficult and costly to identify and resolve problems. On the other hand, enabling too few tracepoints can lead to missing important tracing data, hindering the ability to effectively address the issue at hand [8]. Adaptive tracing is a technique that adjusts the tracing configuration in real-time, leading to an improved trace log collection as well as an efficient process of identifying and resolving issues [9].

Several works have been conducted on adaptive tracing and logging based on static [10]–[12] and dynamic analyses [8], [13]. However, existing works face several limitations. In situations where past unexpected behavior becomes the new expected behavior for a system, existing trend-based adaptive tracing cannot adapt to such increasing or decreasing of the baseline resource usage. Meanwhile, clustering methods would prioritize high standard deviation groups, whereas there could be underlying consistently worse performing offenders going undetected. Also, anomaly-based adaptive tracing would trigger events or tracepoints even for extremely rare occurrences of anomalies which are not always necessary due to their negligible impact on performance.

We introduce a novel three-phase adaptive tracing mechanism that balances the amount of information gathered based on the observed significant changes in program behavior. Our method starts by thoroughly stress-testing the application to gain a deeper understanding of its behavior and identify areas of performance variability and corresponding functions. Then in the production, the method constantly monitors these functions and modifies the tracing configuration (by including or excluding other relevant functions or system calls) as significant changes in behavior are noticed. This results in a more efficient and effective way of collecting just the necessary information for debugging and performance analysis as noise in data is reduced. Our case study assessment indicates that the proposed mechanism is suitable for applications of diverse complexity levels, as shown by its success in effectively gathering essential log data related to various performance issues in real-world systems such as Firefox.

In this work, we present several contributions to the field of adaptive tracing:

- A comprehensive approach to understand program behavior in various stress scenarios and determine the relevant methods, modules, and system calls to monitor.
- An algorithm to group program functions and system calls into different classes and prioritize them based on performance impact.
- An algorithm to adjust tracing configuration by detecting changes in the trend of execution time of program methods and system calls, while filtering out insignificant changes.

The paper is structured into four sections. Section II reviews relevant research in the field. Section III details the proposed methodology for adaptive tracing. Section IV presents a case study on Firefox to showcase the effectiveness of the proposed method. The final section, Section V, provides conclusions and directions for future work.

Authorized licensed use limited to: Brock University. Downloaded on March 06,2026 at 19:37:43 UTC from IEEE Xplore. Restrictions apply.

II. Related Works

Several studies have proposed methods for enhancing adaptive logging and tracing in software systems. Yuan *et al.* [14] suggested static analysis for improving existing logging code. Yao *et al.* [11] and Li *et al.* [15] proposed using deep learning to generate features from source code and adding logging statements. Li *et al.* [10] proposed a deep learning framework to suggest logging locations. Rivera *et al.* [16] proposed injecting logging instructions through Aspect-Oriented Programming.

Dynamic analysis methods were also proposed by Ates *et al.* [8] with a real-time adaptive tracing framework, Zhang *et al.* [13] with K-Means based adaptive instrumentation, and Sun *et al.* [12] with loop identification for online tracing. Dai *et al.* [17] and Yao *et al.* [18] suggested methods for recommending log variables, while Li *et al.* [19] proposed setting log levels using ordinal based neural networks.

Our review of related works highlights limitations in dynamic analysis using techniques of clustering and anomaly-based methods in detecting changes in the normal behavior of the system or application, as well as difficulties in accurately distinguishing between expected and unexpected behavior when there is a shift in normal behavior. These methods also tend to be biased towards groups with smaller values and inadequate in handling changes that were once considered abnormal in the past but are considered normal in the present. Additionally, static analysis-based methods, although effective for general-purpose logging, lack runtime visibility to accurately capture and address performance problems.

III. Methodology

We propose a time-series trend-based adaptive execution tracing framework to address performance issues. Execution tracing can be resource-intensive, so we aim to minimize overhead by selectively tracing only the most relevant methods. Our approach involves: identifying suspicious methods through stress tests, ranking methods based on their probable performance impact, observing the execution time of selected methods, and activating tracepoints to record data upon detecting changes. The architecture of our method is illustrated in Figure 1 and 3 and further explained in subsequent sections.

A. Pre-Testing and Methods Selection

In order to uncover potential performance issues, the application undergoes a stress test, simulating high resource competition and increasing the load on the system CPU, memory, disk, and network. During the stress test, call stack data is collected at regular intervals and analyzed to document all relevant method and system calls. The collected data is then transformed into a time-series representation of execution time, which is examined for trends and patterns that could indicate performance issues.

The method and system calls are grouped into family groups based on their relationship to one another, and the groups are ranked based on the number of calls between the parent and child methods (shown in Figure 2). The groups with the most heavily used methods are prioritized for monitoring.

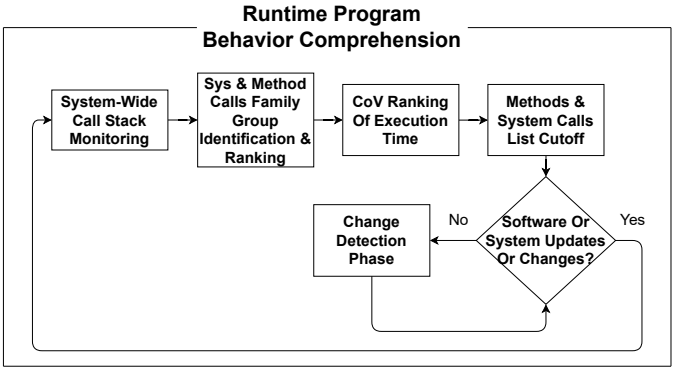

Fig. 1. Pre-Testing, Method Selection & Ranking Methodology

The methods and system calls are also ranked based on the Coefficient of Variation (CoV) of their execution time, considering methods with higher standard deviation to be more likely to cause performance fluctuations. This approach prioritizes monitoring of the most volatile or fluctuating groups in the production environment.

To effectively monitor the performance of a large application, it is necessary to select a limited number of methods to focus on, as monitoring all methods would be too costly. A reasonable cutoff number should be selected based on the overhead costs that we are willing to bear. An example could be to select the top 5% of the methods, upon ranking of method and system calls by their CoV. It is important to periodically re-evaluate the list of methods to monitor, updating it with newly introduced methods and removing depreciated ones, to ensure the relevance and accuracy of the monitoring data in the production environment.

Figure 2 illustrates the Training Phase process. The record function of Perf allows us to directly determine the method that initiated a call to the next method. For this example, we have used a sampling rate of 10 milliseconds. The figure shows that MethodA() called MethodB() in the first call stack, forming a family group of A–B with a count of two. Next, we sort all family groups based on their count in descending order. To find the execution time of MethodA(), we observe that it appears in the first two call stacks but not the third, so we can estimate its execution time to be 20 milliseconds (10ms + 10ms) during its first call. After calculating the execution time for all methods, we calculate their CoV values to rank them based on their fluctuation rate. Finally, we determine a method list cutoff value by only keeping the most volatile methods.

However, although the identified method calls could be correlated with bugs or performance issues, they are not necessarily the main cause of the problems as it could be very well be a normal behavior of such method calls to fluctuate based on the tasks they handle. At this stage, we simply identify the most fluctuating method calls and those contributing the most to latency and mark them as worth monitoring in runtime. It is important to note that our goal is not to detect performance

Authorized licensed use limited to: Brock University. Downloaded on March 06,2026 at 19:37:43 UTC from IEEE Xplore. Restrictions apply.

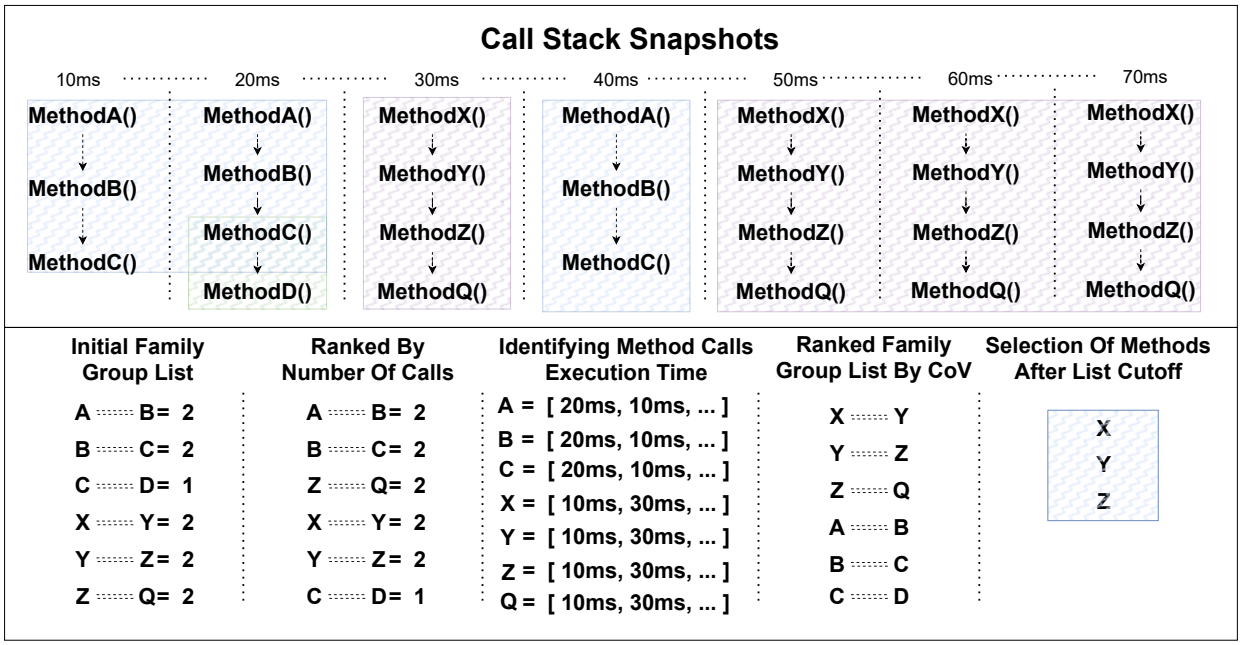

Fig. 2. Family Group Identification & Method Selection For Monitoring

issues at this stage or in runtime. We assume that the program has already been tested functionally and non-functionally and that any bugs have already been identified and fixed. Our objective is to find sufficient candidates to monitor in runtime so that we can detect any significant changes in behavior and use that information for tracing configuration adjustments.

B. Change Detection Phase

In order to change tracing configuration, a decision mechanism has been implemented. The mechanism uses two factors, the detection of changes through Autoregressive Integrated Moving Average (ARIMA) [20] models and a pre-determined threshold value. ARIMA is a statistical method that predicts future values based on historical data. The proposed parameters for the ARIMA model contains the number of lag observations, degree of differencing and the moving average window size to be all the value of 1. A first-order ARIMA(1,1,1) is selected to predict linear trends, where deviations from linearity would indicate potential performance issues. The equation for the model is detailed in [21]. Insignificant changes are filtered out using an anomaly score to avoid unnecessary tracing configuration changes for small, non-threatening changes.

In our proposed framework, the ARIMA models are fed with time-series data constructed from the execution times of the methods and system calls. The models use data up to $time_{t-1}$ to predict the execution time at time $time_t$. If the difference between the actual execution time at $time_t$ and the predicted value exceeds the threshold value, it is determined that there is a deviation from the expected trend, indicating a change. Conversely, if the difference is below the threshold value, it is determined that the change is not significant enough and the function is behaving normally. The threshold is set based on the sensitive nature of the application in question and how tolerable it is to performance deviation. Figure 3 illustrates a summarized flow diagram for the change detection phase.

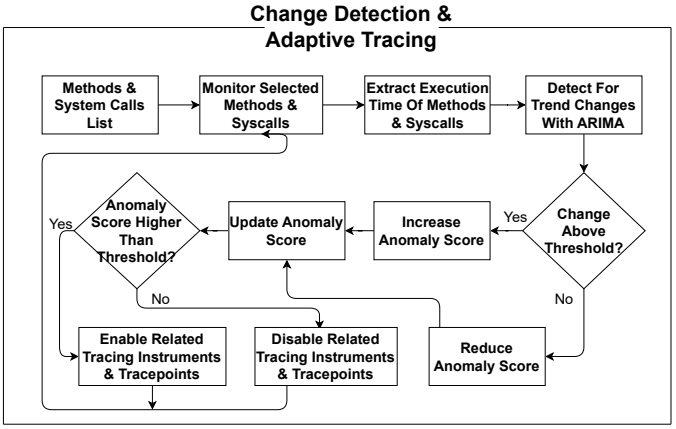

Fig. 3. Change Detection Phase & Adaptive Tracing

The anomaly score function, inspired by a previous work in [22], measures the significance of anomalous execution time data. It assigns a score to methods or system calls based on the frequency of unusual readings and adjusts the score accordingly. A threshold is used to identify potentially affected methods and system calls and add or remove them from the list based on whether the score exceeds or falls below the threshold. The threshold is set based on the specific requirements of the application.

Authorized licensed use limited to: Brock University. Downloaded on March 06,2026 at 19:37:43 UTC from IEEE Xplore. Restrictions apply.

$$anomalyScore = (\beta \times isAnomaly) + ((1-\beta) \times anomalyScore) \quad (1)$$

The β value is a decaying factor that controls the rate of change for the rise or fall of the anomaly score and should be set based on the system's sensitivity or the sensitive nature of the tasks it performs. The $isAnomaly$ is a boolean flag that represents if the execution time reading was anomalous or not (i.e., out of bound from the ARIMA model's prediction).

We preferred the ARIMA method over most machine learning or deep learning techniques because it operates based on past trends and patterns but also adjusts to new stable changes as the moving average shifts. This also makes it easier to detect and respond to any unexpected deviations or breaks in the trend. As a result, ARIMA can provide more accurate and reliable predictions for our use case while avoiding overfitting the recent data. Additionally, ARIMA requires less computational resources compared to other more complex algorithms, making it a more feasible and practical option for our adaptive tracing use case, which needs continuous periodic analysis for its decision-making.

C. Trace Configuration Change

The goal of this phase is to adjust tracing configuration to collect enough data for the early detection of potential performance problems. The proposed multi-level adaptive tracing method enables modifications to trace configurations at both the application and system levels. When the result from the change detection phase is received, the tracing for the associated family groups of the impacted methods will be initiated in this stage, reaching a pre-set depth limit such as 20.

System-level tracing is performed by monitoring the potential suspicious system calls, as determined by the change detection phase and the corresponding kernel events. The Linux system call table [23] is used to map and translate the system call to the corresponding kernel-level component, which can then be targeted for further kernel-level tracing. On the other hand, application-level tracing can be performed directly without any translation needed by enabling tracepoints for the application methods and their family members.

IV. Case Study Results, Evaluation and Discussion

The case study assessment of the method has been successful in recording enough data to address various real-world performance bugs in Firefox such as:

- `Bug 1768481`: Identify issues with driver and application communication.
- `Bug 1637586`: Identify specific methods in application source code causing performance problems for animation rendering.
- `Bug 1776469`: Identify system-level components and system calls responsible for performance problems related to network connectivity and DNS.

The causes of performance issues were successfully found for all the mentioned case studies using our method. In the following, we will describe one of the evaluations mentioned above. This case study was performed on the multi-threaded and complex Firefox web browser to investigate a bug (Bug 1768481) reported in Bugzilla [24] that stated Firefox experiences lags when loading larger PDF files even if they are stored locally. The bug report revealed that the cause of the problem was a particular method call related to VSync, which aligns the refresh rate between the graphics processor and the monitor.

A. Study Setup

In this study, we used the Firefox web browser built from source with the perf flag enabled [25], which enables JIT profiling. The browser was loaded with two different sized PDF files to explore all related methods and system calls. Perf was used to record the call stacks and analyze the data. 8,231 unique family groups of method and system calls were established, sorted and ranked based on the number of calls and execution time. The top 5% (410) of these family groups were selected as the most volatile and impactful for monitoring and performance analysis.

In order to collect the execution time of the method calls, we have used a split of ten-millisecond snapshots while recording call stacks with perf. Figure 2 shows "Identifying Method Calls Execution Time" which is a representation of extracting the execution time from the call stack. This idea was inspired by Microsoft's research named Stackmine [26]. When a method call no longer appears in a consecutive call stack, we assume that the execution time of the method call is the summation of the last continuous blocks of ten milliseconds where it was consecutively visible. We have selected the ten-millisecond window based on the sensitivity of the application. A delay of 10 milliseconds is not so drastic as to cause alerts for a browser application. Our source codes are accessible from our GitHub repository [1].

B. Change Detection

We used an adaptive decision-making process (illustrated in Figure 4) to selectively trace methods only when necessary, resulting in a significant reduction in tracing overhead. In the course of investigating the Bugzilla report [24] of delays in Firefox while loading larger PDF files, a specific method, "mozilla::VsyncRefreshDriverTimer::RunRefreshDrivers," was identified as the cause of the delays, and its execution time was found to exceed 10 milliseconds, with instances exceeding 500 milliseconds observed when loading larger PDFs although stored locally. The family group depth level for tracing was set to 20 which is adjustable based on how granular and detailed data we are willing to collect.

C. Discussion & Limitations

The proposed method has demonstrated its effectiveness in identifying a problematic method in the case study of the Firefox browser. Our method was able to narrow down the number of traced method calls to 410 from a pool of 8,231,

[1] https://github.com/ak19qp/sera2023/

Authorized licensed use limited to: Brock University. Downloaded on March 06,2026 at 19:37:43 UTC from IEEE Xplore. Restrictions apply.

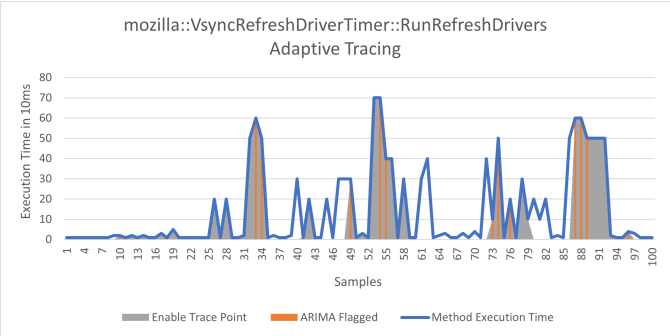

Fig. 4. Adaptive Decision For Tracepoint

TABLE I
CASE STUDY EVALUATION

Evaluation Criteria	Values
Was The Bug Detected?	Yes
Culprit Method	RunRefreshDrivers()
Useful Unique Changes Detected	22%
Percentage Of Samples Traced	56%
Monitoring Overhead Reduced By CoV Ranking	95%
Tracing Overhead Reduced By Change Detection	44%

reducing the tracing overhead significantly. The evaluation criteria and reduction in overhead achieved through ARIMA for change detection are presented in Table I. The results highlight the significant reduction in overhead achieved through the proposed method.

In terms of the informativeness of our method, we could see in Figure 4 that the major changes in the culprit method call were successfully recorded along with 20 methods of its family groups to have sufficient data collected related to the issue. The suggested areas for trace logging show enough earliness of capturing the relevant data to identify the problems. The monitoring of the method calls' execution time using perf caused only an average of 1% or less increase in CPU utilization overhead compared to the baseline (without any monitoring), which is very negligible.

However, we faced a few challenges during the implementation stage of our work. Firstly, the threshold values to define an acceptable fluctuation away from ARIMA as well as the anomaly score's β value had to be manually tested around and selected. Therefore, these values became system and application specific. The threshold value and the β value found from one system wouldn't make sense in another given that it handles different processes and tasks. It was also a challenge to decide on the method list cutoff number. For our case studies, we have used a hard coded 5%, however, in much more complex systems this needs to be addressed along with a dynamic auto-adjustable depth selection for family groups to trace.

V. CONCLUSION & FUTURE WORK

We proposed an adaptive tracing mechanism that leverages the concept of trend detection using autoregressive integrated moving average (ARIMA) models to predict the execution time of application-level methods and system calls. The approach assumes that a change in execution time's trend indicates an underlying performance issue, supported by case study and its results. Our case study results showed that the proposed approach successfully traced and recorded important data related to three different categories of recreated performance problem from Bugzilla reports of Firefox and were able to pinpoint the specific method call responsible for the performance issues.

This research work has demonstrated that the proposed method works for adaptively tracing systems and application performance issues. However, several improvements need to be addressed. We need to introduce an adaptive threshold for the anomaly score, adjusting the ARIMA method's threshold dynamically, handling the method list cutoff value adaptively base on allowable costs, and finding a more accurate way to calculate execution time. We would also explore dynamic cost selection for monitoring the percentage of method calls. These improvements would ensure better adaptive tracing optimality and reduce important data loss. For future work, we would be using LTTng, a tracing tool, which would allow us to execute the tracing configuration suggestions in realtime on tracepoints.

ACKNOWLEDGMENT

The support of the Natural Sciences and Engineering Research Council of Canada (NSERC), Mitacs, Ciena, Brock VPMI and FedDev Ontario is gratefully acknowledged.

REFERENCES

[1] N. Ayewah, W. Pugh, D. Hovemeyer, J. D. Morgenthaler, and J. Penix, "Using static analysis to find bugs," *IEEE software*, vol. 25, no. 5, pp. 22–29, 2008.
[2] T. Ball, "The concept of dynamic analysis," in *Software Engineering—ESEC/FSE'99*. Springer, 1999, pp. 216–234.
[3] D. Yuan, S. Park, and Y. Zhou, "Characterizing logging practices in open-source software," in *2012 34th international conference on software engineering (ICSE)*. IEEE, 2012, pp. 102–112.
[4] B. Cantrill, M. W. Shapiro, A. H. Leventhal *et al.*, "Dynamic instrumentation of production systems." in *USENIX Annual Technical Conference, General Track*, 2004, pp. 15–28.
[5] A. Parker, *Distributed Tracing in Practice*. Van Duuren Media, 2020.
[6] R. Ewaschuk, "Google - Site Reliability Engineering." [Online]. Available: sre.google/sre-book/monitoring-distributed-systems
[7] J. Ellingwood, "An Introduction to Metrics, Monitoring, and Alerting," 12 2017. [Online]. Available: https://www.digitalocean.com/community/tutorials/an-introduction-to-metrics-monitoring-and-alerting
[8] E. Ates, L. Sturmann, M. Toslali, O. Krieger, R. Megginson, A. K. Coskun, and R. R. Sambasivan, "An automated, cross-layer instrumentation framework for diagnosing performance problems in distributed applications," in *Proceedings of the ACM Symposium on Cloud Computing*, 2019, pp. 165–170.
[9] T. M. Peiris and J. H. Hill, "Adapting system execution traces for validation of distributed system qos properties," in *2012 IEEE 15th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing*. IEEE, 2012, pp. 162–171.
[10] Z. Li, T.-H. Chen, and W. Shang, "Where shall we log? studying and suggesting logging locations in code blocks," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, 2020, pp. 361–372.
[11] K. Yao, G. B. De Padua, W. Shang, C. Sporea, A. Toma, and S. Sajedi, "Log4perf: Suggesting and updating logging locations for web-based systems' performance monitoring," *Empirical Software Engineering*, vol. 25, pp. 488–531, 2020.

Authorized licensed use limited to: Brock University. Downloaded on March 06,2026 at 19:37:43 UTC from IEEE Xplore. Restrictions apply.

[12] H. Sun, C. Zhang, H. Li, Z. Wu, L. Wu, and Y. Li, "Atos: Adaptive program tracing with online control flow graph support," *IEEE Access*, vol. 7, pp. 127 495–127 510, 2019.
[13] S. Zhang, D. Liu, L. Zhou, Z. Ren, and Z. Wang, "Diagnostic framework for distributed application performance anomaly based on adaptive instrumentation," in *2020 2nd International Conference on Computer Communication and the Internet (ICCCI)*. IEEE, 2020, pp. 164–169.
[14] D. Yuan, J. Zheng, S. Park, Y. Zhou, and S. Savage, "Improving software diagnosability via log enhancement," *ACM Transactions on Computer Systems (TOCS)*, vol. 30, no. 1, pp. 1–28, 2012.
[15] J. Li, P. He, J. Zhu, and M. R. Lyu, "Software defect prediction via convolutional neural network," in *2017 IEEE international conference on software quality, reliability and security (QRS)*. IEEE, 2017, pp. 318–328.
[16] F. Rivera-Ortiz and L. Pasquale, "Towards automated logging for forensic-ready software systems," in *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*. IEEE, 2019, pp. 157–163.
[17] S. Dai, Z. Luan, S. Huang, C. Fung, H. Wang, H. Yang, and D. Qian, "Reval: Recommend which variables to log with pre-trained model and graph neural network," *IEEE Transactions on Network and Service Management*, 2022.
[18] C. Yao, D. Agrawal, G. Chen, B. C. Ooi, and S. Wu, "Adaptive logging: Optimizing logging and recovery costs in distributed in-memory databases," in *Proceedings of the International Conference on Management of Data*, 2016, pp. 1119–1134.
[19] Z. Li, H. Li, T.-H. Chen, and W. Shang, "Deeplv: Suggesting log levels using ordinal based neural networks," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1461–1472.
[20] S. L. Ho and M. Xie, "The use of arima models for reliability forecasting and analysis," *Computers & industrial engineering*, vol. 35, no. 1-2, pp. 213–216, 1998.
[21] B. K. Nelson, "Time series analysis using autoregressive integrated moving average (arima) models," *Academic emergency medicine*, vol. 5, no. 7, pp. 739–744, 1998.
[22] J. Ehlers, A. van Hoorn, J. Waller, and W. Hasselbring, "Self-adaptive software system monitoring for performance anomaly localization," in *Proceedings of the 8th ACM international conference on Autonomic computing*, 2011, pp. 197–200.
[23] F. Valsorda, "Searchable linux syscall table for x86 and x86_64 — pytux." [Online]. Available: https://filippo.io/linux-syscall-table/
[24] clessili, "Bug 1768481," 5 2022. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=1768481
[25] Mozilla, "Jit profiling with perf — firefox source docs documentation." [Online]. Available: https://firefox-source-docs.mozilla.org/performance/jit_profiling_with_perf.html
[26] S. Han, Y. Dang, S. Ge, D. Zhang, and T. Xie, "Performance debugging in the large via mining millions of stack traces," in *2012 34th International Conference on Software Engineering (ICSE)*. IEEE, 2012, pp. 145–155.

Authorized licensed use limited to: Brock University. Downloaded on March 06,2026 at 19:37:43 UTC from IEEE Xplore. Restrictions apply.